ngspice Linear Solvers — KLU vs. Sparse 1.3

Behavior, defaults, and limitations across DC / AC / transient / noise

Ngspice + OpenVAF Enhancements project

July 2026

Contents

ngspice linear solvers — KLU vs. Sparse 1.3 (behavior, defaults, limitations)

This build of ngspice-46 ships two direct linear solvers, and they are not fully interchangeable across analysis types. This note records exactly how they differ — which is the default, how to select and confirm each, and where KLU falls short — based on a direct read of the source and a solver-by-solver sweep of the whole `examples/` suite.

TL;DR

	KLU	Sparse 1.3
Availability in this build	Compiled in (SuiteSparse, statically linked — 44 `klu_*` symbols in the binary)	Always present (ngspice's own solver)
Default?	No	Yes — this build defaults to Sparse 1.3
How to select	`.option klu`	`.option sparse` (or just the default)
DC op / DC sweep	✓ correct	✓ correct
AC	✓ correct	✓ correct
Transient	✓ correct (one caveat below)	✓ correct
Noise (**`.noise`**)	✓ correct (since E-113)	✓ correct
Pole-zero (**`.pz`**)	✓ single-ended; ⚠balanced-output Sparse-only	✓ correct
Sensitivity (**`.sens`**, DC & AC)	✓ correct (since E-114)	✓ correct
Distortion (**`.disto`**)	✓ correct (since E-115)	✓ correct
Periodic steady state (**`.pss`**)	× hangs — guarded to Sparse (E-117)	✓ correct

Practical guidance: leave the default (Sparse 1.3) unless you have a specific reason to switch. Sparse 1.3 runs every analysis in the suite. Since Enhancement-113 KLU also runs noise and single-ended pole-zero correctly, and since Enhancement-114 it runs DC/AC sensitivity, and since Enhancement-115 distortion (**`.disto`**) correctly; the only analysis still Sparse-only under KLU is balanced-output pole-zero. Reach for `.option klu` on large, sparse DC/AC problems where KLU's ordering and factorization are faster, and expect it to be less robust on stiff transient edges (see the one genuine discrepancy below).

Selecting and confirming the solver

The netlist option is a simple flag:

```
.option klu       * use KLU (SuiteSparse)
.option sparse    * use the legacy Sparse 1.3  (this is also the default)
```

Because unknown `.option` keywords are silently ignored by ngspice, "it ran without complaint" is *not* evidence the solver changed. To confirm which solver is actually active, use the interactive `option` command, which prints the live `CKTkluMODE`:

```
.control
run
option        * prints, among other things:  "Matrix solver:  KLU"  or  "Sparse 1.3"
.endc
```

On the committed `bin/` binary this reports **`Sparse 1.3`** for a bare deck and for `.option sparse`, and **`KLU`** only when `.option klu` is given — confirming Sparse 1.3 is the default here.

Noise and pole-zero under KLU (Enhancement-113)

ngspice used to refuse both outright (`noisean.c` / `pzan.c` printed "not (yet) supported with 'option KLU'"). The real reason for noise was subtler than "not wired up": noise uses the adjoint method — it solves the *transposed* system $A^T \cdot x = e$ via `SMPcaSolve` — and `SMPcaSolve`'s KLU branch was calling the non-transposed `klu_z_solve` where its Sparse branch calls `spSolveTransposed`. That silently produced the wrong noise for any asymmetric matrix (every circuit with a transistor or controlled source), which is why it was disabled rather than merely slow.

Enhancement-113 fixes the adjoint solve (`klu_z_tsolve`) and lifts the guards, so under KLU:

- Noise is correct — verified identical to Sparse on resistor thermal noise, asymmetric VCCS circuits, OSDI device models, and integrated totals. (`.sp` S-parameters share the same adjoint solve and are corrected too.)
- Single-ended pole-zero (grounded output reference) runs correctly.
- Balanced-output pole-zero (non-grounded reference) stays Sparse-only: its zeros phase folds columns at solve time, which Sparse survives via dynamic Markowitz re-ordering but KLU's fixed symbolic factorization cannot; a targeted guard now directs that case to `.option sparse`.

Enhancement-114 then fixes sensitivity under KLU. Sensitivity builds an auxiliary perturbation matrix `delta_Y` that is a plain Sparse matrix (it is only multiplied, never factored); two KLU setup blocks were gated on the *main* matrix's flag and dereferenced the NULL `delta_Y->SMPkluMatrix`, segfaulting on every DC/AC `.sens` deck. Gating them on `delta_Y`'s own flag keeps it Sparse under KLU too, so both DC and AC sensitivity now match Sparse exactly.

- Distortion (**`.disto`**) was Sparse-only — `distoan.c` had no KLU code, so under `.option klu` the (complex) distortion solves ran against a matrix left in real mode and silently produced no output. Surfaced while validating E-114. Enhancement-115 fixes it: it converts the KLU matrix real↔complex around the distortion solve loop (exactly as `acan.c` does for AC), so `.disto` now matches Sparse bit-for-bit.

One genuine KLU discrepancy (opamp741)

Beyond balanced-output pole-zero, exactly one example still produces a different (wrong) result under KLU than under Sparse; it is marked `KLU_XFAIL` in the dual-solver harness so it is exercised and tracked rather than silently skipped.

opamp741 — stiff transient diverges. The transistor-level opamp741 example (a μA741 from ~70 lines of Verilog-A BJT):

- Sparse 1.3: the large-signal slew test (`pulse(-5 5 …)` into the follower) runs the full `tran 20n 80u` cleanly (~4058 points), slew rate ≈ 0.54 V/μs.
- KLU: the same run diverges at the slewing edge — output-stage transistors switch off, their transconductances collapse, KLU declares the Jacobian singular (nodes `x1.o1/o2/b34/cm`), the timestep collapses, and ngspice aborts at t ≈ 2.03 μs (~133 points).

A convergence/robustness difference on a stiff circuit: KLU's fill-reducing symbolic ordering is computed once and `klu_factor` can only pivot *within* it, whereas Sparse re-orders every factorization (dynamic Markowitz threshold pivoting) and survives the fast edge. This was confirmed not fixable with the available knobs — full partial pivoting (`tol = 1.0`), disabling BTF and re-analyzing a fresh ordering, and the gmin-loading path (identical to Sparse — both skip absent diagonals) all left the abort unchanged (E-116). A real fix would need a hybrid solver that falls back

to Sparse when KLU's factorization fails. (E-111's line search does not help either — it damps the DC-op Newton, not per-timepoint transient iterations.)

Two former discrepancies, now fixed (E-116). `groundcontrib` (node-to-ground `V(p,gnd) <+ 1.5` read `v(p)=0` under KLU instead of `1.5`) and `hierbranch` (hierarchical branch-*current* probes read `0`) were both structural, not numerical: an OSDI internal node that appears in no Jacobian entry — a `ground` reference whose branch contribution drops its column — was allocated its own all-zero solver row, which Sparse tolerates but KLU treats as structurally singular. Enhancement-116 ties such decoupled internal nodes to ground at setup, so both now match Sparse under KLU.

The dual-solver test harness

Every `verify_*.py` in `examples/` is run under both solvers automatically. Each script calls `check_both_solvers(__file__)` (right after importing `_setup`); on a normal run that re-executes the script once per solver — injecting `.option sparse` / `.option klu` into every ngspice deck — and prints a combined verdict:

```
=== BOTH-SOLVER RESULT [ceil]: sparse=PASS  klu=PASS => OK ===
```

KLU's one remaining unsupported analysis (balanced-output pole-zero) is auto-detected and reported `SKIP`; the single `KLU_XFAIL` example above (`opamp741`) is expected-fail under KLU. Env escape hatches: `NGSPICE_SOLVER=klu|sparse` runs once under one solver; `NG_BOTH=0` disables the dual run.

Sweep result (all 101 machine-checkable scripts): 101/101 OK — every example is `sparse=PASS` with `klu` in `{PASS, SKIP, XFAIL}`, i.e. the two solvers agree wherever KLU is applicable. (The `linesearch` example initially *crashed* under KLU, which Enhancement-112 fixed; Enhancement-113 then moved 10 examples from KLU-skipped to KLU-passing by enabling noise and single-ended pole-zero; Enhancement-114 fixed the AC/DC-sensitivity crash under KLU, and — by fixing the deck-injector restore bug — un-masked two more `XFAIL`s (`groundcontrib`, `hierbranch`); Enhancement-115 then fixed distortion, so the `analyses` example now runs fully under KLU; and Enhancement-116 fixed `groundcontrib` and `hierbranch` at their shared root cause. The only remaining `XFAIL` is `opamp741`.)

Conclusion: for DC, AC, transient, noise, single-ended pole-zero, sensitivity, and distortion the two solvers agree across the suite, with exactly one KLU exception (the stiff `opamp741` transient), while balanced-output pole-zero is Sparse-1.3-only by ngspice design. Sparse 1.3, the default, covers everything — which is why it is the default and why the dual-solver harness keys correctness off it.